🚀 Modernise & Migrate: DWER TravelReporting — Data, Analytics & Reporting Track

Welcome to **Day 2 (Build & Migrate)** and **Day 3 (Harden & Deliver)** of the Application Modernisation OpenHack for the Department of Water and Environmental Regulation (DWER). Day 1 (Discovery & Orientation) is already complete.

Over the next two days you will transform a 20-year-old Oracle database and a set of ColdFusion-rendered HTML reports into a modern, cloud-native data platform: Azure SQL as the operational store, Microsoft Fabric with Bronze/Silver/Gold Lakehouses for analytics, and a Power BI semantic model version-controlled as TMDL — all deployed via automated CI/CD pipelines.

Why this track?

Data Engineers, BI Developers, and Analytics Specialists are the people who turn raw operational data into organisational insight. The TravelReporting system currently stores all its data in a single Oracle schema with no analytics layer, no semantic model, and no automated reporting — travel reports are hand-crafted HTML pages generated by ColdFusion scripts. This track gives you the opportunity to design and build a proper data platform from scratch, using GitHub Copilot to accelerate schema migration, medallion architecture design, DAX measure generation, and pipeline automation. By the end you will have a repeatable playbook for modernising legacy Oracle data estates onto Microsoft Fabric with AI assistance.

Your Tools

GitHub Copilot Chat Modes

Mode	When to use	Think of it as…
💬 **Ask**	Understanding Oracle-specific DDL, explaining migration risks, reviewing query plans, asking "what does this DECODE do?"	Your senior DBA on call
📋 **Plan**	Designing the medallion architecture, mapping ColdFusion reports to Power BI visuals, planning the ETL topology before writing a single pipeline	Your data architect writing the blueprint
🤖 **Agent**	Generating target Azure SQL DDL, writing PySpark notebooks, creating DAX measures, scaffolding Fabric pipelines, building DACPAC CI/CD	Your data engineer who never sleeps

> **Rule of thumb:** Ask → Plan → Agent. Understand the Oracle schema before planning the medallion layout, plan the medallion layout before generating notebooks.

Data & Analytics Accelerators

Tool	When it helps
Azure MCP Server	Query Azure SQL metadata, explore OneLake storage, inspect Fabric workspace resources, validate deployments — without leaving VS Code
Power BI MCP Server	Author semantic models, generate DAX measures, export TMDL, manage datasets (`mcp_powerbi-model_*`) — see learn.microsoft.com/power-bi/developer/mcp
Microsoft Fabric MCP Server	Scaffold lakehouses, create notebooks, manage Fabric pipelines and workspaces (`mcp_fabric_mcp_*`) — see the Fabric VS Code extension
Azure Skills for GitHub Copilot	`@azure-prepare` , `@azure-storage` , `@azure-kusto` , `@azure-cost` , `@azure-resource-visualizer` — invoke Azure data operations via Copilot
Awesome Copilot	Reusable `.prompt.md` templates for SQL migration, DAX, KQL, and notebook scaffolding at github.com/github/awesome-copilot

The Data Estate

Source: Oracle `TRRE` Schema

The TravelReporting application is backed by an Oracle database under the `TRRE` schema. All DDL is available in `TRRE-Oracle/` .

Dimension	Details
Engine	Oracle (version not pinned in DDL; NLS_COMP collation, `RESULT_CACHE` , `NO INMEMORY` clauses suggest 12c or later)
Schema owner	`TRRE`
DDL assets	`TRRE-Oracle/TABLE.sql` , `TRRE-Oracle/VIEW.sql` , `TRRE-Oracle/FUNCTION.sql` , `TRRE-Oracle/SEQUENCE.sql` , `TRRE-Oracle/INDEX.sql` , `TRRE-Oracle/CONSTRAINT.sql`

Tables

Table	Description	Migration notes
`TRRE.TRAVEL`	Core entity — ~60 columns covering travel proposals, approval dates, costs (estimated + actual), financial coding, and traveller details	Largest table; many nullable fields; `NUMBER` → `decimal` / `int` mapping needed
`TRRE.ACCOUNT_CODES`	Lookup: financial account codes	Low risk
`TRRE.ARRANGER_GROUP`	Lookup: groups of travel arrangers	Low risk
`TRRE.ARRANGER_USER`	Junction: arranger ↔ group	Low risk

Table	Description	Migration notes
`TRRE.APPROVER_USER`	Junction: approver ↔ approval area	Low risk
`TRRE.FLEX_COST_CODE`	Financial flex coding (cost centre, fund, account, activity)	Oracle NUMBER columns; review precision
`TRRE.FLEX_PROJ_CODE`	Financial flex coding (project type, fund, account)	As above
`TRRE.FUNDING_TYPE`	Lookup: funding type	Low risk
`TRRE.SYSTEM_REF_VALUES`	Domain reference values (travel type, class, status etc.)	Replaces Oracle `DECODE` lookups

Oracle-specific features requiring translation

Feature	Location	Azure SQL equivalent
`DECODE(col, val, result, ...)`	Queries throughout `TravelReporting/` CFM files	`CASE WHEN … THEN … END`
`SYSDATE`	All `INSERT_DATE`, `UPDATE_DATE` columns	`GETDATE()` / `SYSDATETIME()`
`NLS_COMP` `USING_NLS_COMP` collation	All `VARCHAR2` columns in `TABLE.sql`	`Latin1_General_CI_AS` or `SQL_Latin1_General_CP1_CI_AS`
Oracle sequences (`SEQUENCE.sql`)	`TRAVEL.ID`, `ARRANGER_GROUP.ID`, etc.	`IDENTITY(1,1)` or sequences via `CREATE SEQUENCE`
PL/SQL functions (`FUNCTION.sql`)	Business logic embedded in the DB	T-SQL scalar/table-valued functions or EF Core computed properties
Cross-schema views	`TRRE-Oracle/VIEW.sql` — `V_STAFF_CUR` (from `COMA`), `V_COSM_STRUCTURE` (from `COSM`), `V_PROJECT_CURRENT` (from `BUSY`)	Seed data or external lookup tables in Azure SQL

Reporting assets (source)

Report	Source file	Analytics target
Monthly travel report	`TravelReporting/Reports/Monthly/Display.cfm`	Power BI report page — monthly cost trend
Quarterly travel report	`TravelReporting/Reports/Quarterly/Display.cfm`	Power BI report page — quarterly summary
Yearly travel report	`TravelReporting/Reports/Yearly/`	Power BI report page — annual travel analysis

Report	Source file	Analytics target
Division report	`TravelReporting/Reports/Custom/`	Power BI report page — by division/org unit
Monthly graph	`TravelReporting/Graphs/Monthly/Display.cfm`	Power BI visual — line/bar chart
Yearly graph	`TravelReporting/Graphs/Yearly/Display.cfm`	Power BI visual — annual comparison
Division graph	`TravelReporting/Graphs/Division/Display.cfm`	Power BI visual — division breakdown
Custom search results	`TravelReporting/Search/ReportDisplay.cfm`	Power BI paginated report or interactive table

ETL / batch patterns

No SSIS packages, scheduled jobs, or ETL tooling found. All data flows through ColdFusion web forms into Oracle via inline SQL. The target Fabric pipelines will be the first formal ETL layer for this workload.

Target Data Platform

Layer	Target
Operational store	Azure SQL — `TravelReporting` database, `trre` schema
Lakehouse	Microsoft Fabric — Bronze / Silver / Gold lakehouses in a dedicated workspace
Ingestion	Fabric Data Pipeline (Azure SQL → Bronze)
Transformation	Fabric Notebooks (PySpark / SQL) — Bronze → Silver → Gold
Semantic model	Power BI semantic model (team's choice: Power BI MCP + TMDL, Power BI Desktop `.pbix`, or Direct Lake on Fabric)
Reporting	Power BI reports replacing ColdFusion HTML reports
DB schema CI/CD	DACPAC + `sqlpackage` + Azure DevOps

Agenda

Day	Phase	Topic	Time
Day 2	1	DB Migration Assessment & Schema Scripts	60 min
Day 2	2	Test / Dummy Data Generation & Validation	60 min

Day	Phase	Topic	Time
Day 2	3	Power BI Semantic Model & DAX Measures	60 min
Day 2	4	Fabric Lakehouse & Medallion Architecture	60 min
Day 2	5	Pipeline & Notebook Identification	60 min
Day 3	6	Performance Tuning & Index Optimisation	60 min
Day 3	7	Migration Automation (ETL/ELT) & DB CI/CD	60 min
Day 3	8	Notebooks, Fabric Pipelines, TMDL Export, DR Scripts & Solution Docs	60 min

The Phases

Phase 1 — DB Migration Assessment & Schema Scripts

Ask + Agent mode

The TRRE Oracle schema is the authoritative record of the TravelReporting data model. Before writing a single line of T-SQL, you need to understand what Oracle-specific constructs exist, which ones have direct Azure SQL equivalents, and which ones require re-engineering. This phase produces the migration assessment and the target Azure SQL DDL — the foundation everything else builds on.

Your Challenge

Inventory the Oracle schema, classify every object by migration risk, and generate target Azure SQL DDL that is functionally equivalent and free of Oracle-specific syntax.

Things to Explore/Do

- Open `TRRE-Oracle/TABLE.sql`. Ask Copilot: "*Review this Oracle DDL. For each table, list: (1) the Oracle-specific features used (NUMBER precision, VARCHAR2, NLS_COMP, RESULT_CACHE, NO INMEMORY), (2) the Azure SQL / T-SQL equivalent for each feature, and (3) a migration risk rating (Low / Medium / High) with justification.*"
- Ask Copilot to generate a full Azure SQL DDL script for `TRRE.TRAVEL` that:
  - Replaces `NUMBER` with appropriate `INT`, `DECIMAL`, or `BIGINT`
  - Replaces `VARCHAR2(n BYTE)` with `NVARCHAR(n)`
  - Replaces Oracle sequences with `IDENTITY(1,1)` or `CREATE SEQUENCE`
  - Sets collation to `Latin1_General_CI_AS` (or let Copilot suggest the best match for government data)
  - Preserves all nullability constraints visible in `TRRE-Oracle/CONSTRAINT.sql`
- Open `TRRE-Oracle/VIEW.sql`. Ask Copilot: "*The views V_STAFF_CUR, V_COSM_STRUCTURE, and V_PROJECT_CURRENT reference Oracle schemas COMA, COSM, and BUSY that will not be available in Azure SQL. For each view, describe what data it provides and suggest how to replace it: (a) seed table, (b) external REST lookup, or (c) Entra ID directory query.*"

- Open `TRRE-Oracle/FUNCTION.sql`. Ask Copilot: "*Translate these Oracle PL/SQL functions to T-SQL scalar or table-valued functions for Azure SQL.*"
- Open `TRRE-Oracle/INDEX.sql`. Ask Copilot: "*Review these Oracle indexes. Which ones translate directly to SQL Server? Are there any bitmap or function-based indexes that need a different approach in Azure SQL?*"
- Ask Copilot to generate a **migration risk matrix** as a Markdown table: each Oracle object, its type, risk level, and the specific action required (translate, rewrite, stub, retire).
- Use `@azure-prepare` to confirm the Azure SQL server from the platform track (Phase 1 of that guide) is provisioned and the `trre` schema can be created.

What Done Looks Like

- ☐ Migration risk matrix exists for all objects in `TRRE-Oracle/` (tables, views, functions, sequences, indexes, constraints)
- ☐ Target Azure SQL DDL exists for all 9 `TRRE` tables under `database/migrations/`
- ☐ Oracle-specific syntax (`DECODE`, `SYSDATE`, `VARCHAR2`, `NUMBER`, NLS_COMP, sequences) is fully translated
- ☐ Cross-schema view replacement strategy is documented (seed tables chosen for hackathon scope)
- ☐ PL/SQL functions are translated to T-SQL equivalents

🌟 **Stretch Goal** — Use Agent mode to generate a `database/migrations/001_initial_schema.sql` file structured as a DACPAC-compatible migration script, ready for `sqlpackage` deployment in Phase 7.

⏸️ GIT CHECKPOINT

```
git add database/ && git commit -m "phase-1: Oracle migration assessment complete — Azure SQL DDL
```

Phase 2 — Test / Dummy Data Generation & Validation

🤖 Agent mode

Real migration testing requires data. The Oracle database will not be available in the hackathon environment, and production data must never be used for development. This phase generates realistic, referentially-consistent sample data for the Azure SQL target schema, plus validation scripts that can be run after any data load to confirm completeness and correctness.

Your Challenge

Generate a full set of FK-safe sample data for the Azure SQL `trre` schema and write row-count and checksum validation scripts suitable for pipeline use.

Things to Explore/Do

- Use Agent mode: "*Generate a T-SQL seed data script for the following Azure SQL tables in dependency order (lookup tables first, then junction tables, then TRAVEL): SYSTEM_REF_VALUES, FUNDING_TYPE, ACCOUNT_CODES, ARRANGER_GROUP, ARRANGER_USER, APPROVER_USER, FLEX_COST_CODE, FLEX_PROJ_CODE, TRAVEL. Generate at*

least 5 rows per lookup table and 50 rows in TRAVEL. Use realistic DWER-style data: WA destinations, government cost centres, realistic travel costs."

- Ask Copilot to generate seed rows for `SYSTEM_REF_VALUES` that cover all domains found in the ColdFusion queries:
  - `TRAVEL_TYPE` (e.g. Domestic, Interstate, International)
  - `TRAVEL_CLASS` (e.g. Economy, Business)
  - `STATUS` (e.g. Proposed, Approved, Completed, Cancelled)
- Ask Copilot to generate surrogate staff records for the cross-schema views: a seed table `ref.staff` with `NT_LOGIN`, `FIRST_NAME`, `SURNAME`, `WORK_EMAIL`, `ORG_CODE` columns replicating the shape of `V_STAFF_CUR` (`TRRE-Oracle/VIEW.sql`).
- Use Agent mode: "*Generate a T-SQL validation script that: (1) asserts row counts in each table match expected minimums after seeding, (2) asserts no orphaned FK references exist in TRAVEL (ARRANGER_GROUP, FLEX_COST_CODE etc.), (3) asserts TRAVEL.TOTAL_COST equals the sum of cost components for each row, (4) prints PASS/FAIL per check.*"
- Ask Copilot: "*Generate a PySpark cell that reads the TRAVEL table from Azure SQL, computes a SHA-256 checksum of the sorted dataset, and prints it — so we can compare source vs. target after a migration run.*"
- Store seed scripts under `database/seeds/` and validation scripts under `database/validate/`.

What Done Looks Like

- ☐ Seed scripts produce at least 50 `TRAVEL` rows and all lookup table rows without FK violations
- ☐ `SYSTEM_REF_VALUES` covers `TRAVEL_TYPE`, `TRAVEL_CLASS`, and `STATUS` domains
- ☐ Surrogate staff/org reference data replaces the cross-schema view data
- ☐ Validation script checks row counts, FK integrity, and at least one business-rule assertion
- ☐ All scripts are idempotent (can be re-run without error)
- ☐ Scripts are stored under `database/seeds/` and `database/validate/`

🌟 **Stretch Goal** — Use the Fabric MCP to upload the seed data as Parquet files to the Bronze lakehouse, so Phase 4 has real data to process from Day 1.

Phase 3 — Power BI Semantic Model & DAX Measures

📋 Plan + 🤖 Agent mode

The TravelReporting system currently delivers reports as static HTML pages generated by ColdFusion (`TravelReporting/Reports/Monthly/Display.cfm`, `Quarterly`, `Yearly`, `Custom`). These pages have no drill-through, no filtering, no mobile layout, and are regenerated on every page load from live Oracle queries. This phase replaces them with a proper Power BI semantic model and interactive reports. Your team chooses its preferred authoring approach.

Your Challenge

Build a Power BI semantic model from the migrated Azure SQL schema, with DAX measures that reproduce the reporting logic from the ColdFusion reports, and at least two interactive report pages.

Things to Explore/Do

Choose your authoring lane — all three options produce the same deliverable:

Option A — Power BI MCP + TMDL (recommended for source-control-first teams):

- Use `mcp_powerbi-model_create_semantic_model` to create a new semantic model connected to the Azure SQL `trre` schema.
- Ask Copilot: "*Use the Power BI MCP to define tables for TRAVEL, ACCOUNT_CODES, ARRANGER_GROUP, FUNDING_TYPE, and SYSTEM_REF_VALUES. Add relationships: TRAVEL[ARRANGER_GROUP] → ARRANGER_GROUP[ID], TRAVEL[ACCOUNT_CODE] → ACCOUNT_CODES[ACCOUNT_CODE].*"
- Use `mcp_powerbi-model_export_tmdl` to export the model as TMDL files into `powerbi/semantic-model/` for source control.

Option B — Power BI Desktop (.pbix):

- Connect to the Azure SQL `trre` schema using DirectQuery or Import mode.
- Use Copilot in VS Code to generate DAX measures, then paste them into the Desktop measure editor.
- Export the `.pbix` file to `powerbi/` in the repository *(note: `.pbix` is binary — see Tips for why TMDL is preferred)*.

Option C — Direct Lake on Fabric:

- Connect the semantic model directly to the Gold lakehouse Delta tables (created in Phase 4).
- Use `mcp_powerbi-model_*` tools or the Fabric portal to create the model in Direct Lake mode.
- Export as TMDL via the Power BI MCP.

DAX measures (all teams):

Use Agent mode to generate the following DAX measures, grounded in the columns of `TRRE.TRAVEL` (`TRRE-Oracle/TABLE.sql`):

```
Total Estimated Cost = SUM(TRAVEL[TOTAL_COST])
Total Actual Cost = SUM(TRAVEL[ACT_TOTAL_COST])
Cost Variance = [Total Actual Cost] - [Total Estimated Cost]
Cost Variance % = DIVIDE([Cost Variance], [Total Estimated Cost])
Travel Count = COUNTROWS(TRAVEL)
MTD Travel Count = CALCULATE([Travel Count], DATESMTD(TRAVEL[DEPART_DATE]))
YTD Travel Cost = CALCULATE([Total Actual Cost], DATESYTD(TRAVEL[DEPART_DATE]))
MoM Cost Change % =
    VAR PrevMonth = CALCULATE([Total Actual Cost], PREVIOUSMONTH(TRAVEL[DEPART_DATE]))
    RETURN DIVIDE([Total Actual Cost] - PrevMonth, PrevMonth)
Avg Cost Per Trip = DIVIDE([Total Actual Cost], [Travel Count])
Airfare % of Total = DIVIDE(SUM(TRAVEL[ACT_AIRFARE_COST]), [Total Actual Cost])
```

Report pages to create (replacing ColdFusion reports):

- **Monthly Cost Trend** (replacing `TravelReporting/Reports/Monthly/Display.cfm`) — bar/line chart of actual cost by month, sliced by `TRAVEL_TYPE` and `STATUS`
- **Division Summary** (replacing `TravelReporting/Reports/Custom/Display.cfm`) — cost and count by organisational unit (using the `PERSON_ORG_CODE` / org reference data)

What Done Looks Like

- ☐ Semantic model is connected to the Azure SQL `trre` schema
- ☐ All 10 DAX measures above are implemented and return correct results against seed data
- ☐ At least 2 report pages are created (Monthly Cost Trend, Division Summary)
- ☐ Report includes slicers for date range, travel type, status, and division
- ☐ Semantic model is exported as TMDL into `powerbi/semantic-model/` (or `.pbix` if Option B chosen, with a note)

🌟 **Stretch Goal** — Add a **calculation group** called "Time Intelligence" with calculation items for MTD, QTD, YTD, and Prior Year — so any measure in the model gets time-intelligence variants without duplicating DAX.

Phase 4 — Fabric Lakehouse & Medallion Architecture

📋 Plan + 🤖 Agent mode

Operational reporting directly on Azure SQL is fine for current-state queries, but DWER will eventually want historical trend analysis, cross-system joins (travel + finance + HR), and self-service analytics that should never run against the production database. Microsoft Fabric with a Bronze/Silver/Gold medallion architecture provides the analytics layer that sits alongside Azure SQL — ingesting data from the operational store, transforming it into analytics-ready Gold tables, and serving the Power BI semantic model.

Your Challenge

Scaffold a Microsoft Fabric workspace with three lakehouses (Bronze, Silver, Gold), define the ingestion contract from Azure SQL, and document the medallion layout with table-level lineage.

Things to Explore/Do

- Use the Fabric MCP to scaffold the workspace: `mcp_fabric_mcp_create_workspace` — create a workspace called `TravelReporting-Analytics`.
- Use `mcp_fabric_mcp_create_lakehouse` to create three lakehouses:
  - `bronze_travel` — raw ingest from Azure SQL, no transformation, schema-on-read
  - `silver_travel` — cleaned, typed, deduplicated Delta tables
  - `gold_travel` — aggregated, business-rule-applied tables ready for Power BI Direct Lake
- In Plan mode, ask Copilot: "*Design the Bronze/Silver/Gold table layout for the TravelReporting data estate. Source tables are: TRAVEL (~60 cols), ACCOUNT_CODES, ARRANGER_GROUP, ARRANGER_USER, APPROVER_USER, FLEX_COST_CODE, FLEX_PROJ_CODE, FUNDING_TYPE, SYSTEM_REF_VALUES. What transformations happen at each layer? What is the Gold schema — what aggregated/denormalised tables does Power BI need?*"
- Document the medallion layout in `docs/medallion-architecture.md` as a Mermaid diagram:

```
graph LR
  AzureSQL[(Azure SQL\ntrre schema)] -->|Fabric Pipeline| B[bronze_travel\nRaw Delta tables]
  B -->|Notebook: bronze→silver| S[silver_travel\nCleaned Delta tables]
  S -->|Notebook: silver→gold| G[gold_travel\nAggregated Delta tables]
  G -->|Direct Lake| PBI[Power BI\nSemantic Model]
```

- Ask Copilot to design the Gold layer table `gold.travel_monthly_summary` that pre-aggregates the data needed for the Monthly Cost Trend report (replacing `TravelReporting/Reports/Monthly/Display.cfm`):
  - `year`, `month`, `travel_type`, `status`, `org_code`
  - `total_estimated_cost`, `total_actual_cost`, `trip_count`, `avg_cost`
- Use `mcp_fabric_mcp_list_lakehouses` to verify all three lakehouses are created in the workspace.
- Use `@azure-storage` to confirm OneLake is accessible and the lakehouse Delta table paths are correct.

What Done Looks Like

- ☐ Fabric workspace `TravelReporting-Analytics` exists with three lakehouses
- ☐ Bronze, Silver, and Gold table schemas are designed and documented
- ☐ `docs/medallion-architecture.md` has a Mermaid lineage diagram with all source tables, layers, and the Power BI connection
- ☐ `gold.travel_monthly_summary` schema is defined (DDL or notebook cell)
- ☐ Fabric MCP confirms lakehouses are provisioned

🌟 **Stretch Goal** — Define a data quality contract for the Silver layer: a Notebook cell that asserts `TRAVEL.TOTAL_COST IS NOT NULL`, `TRAVEL.DEPART_DATE ≤ TRAVEL.RETURN_DATE`, and `TRAVEL.STATUS IN ('Proposed','Approved','Completed','Cancelled')` — failing the notebook run if any assertion is violated.

⏸️ GIT CHECKPOINT

```
git add database/ docs/ powerbi/ notebooks/ && git commit -m "phase-4: Fabric workspace scaffolde
```

Phase 5 — Pipeline & Notebook Identification

💬 Ask + 📋 Plan mode

The TravelReporting system has no ETL layer — data flows only through web forms. That means you are starting with a blank sheet for the analytics pipeline design. This phase catalogues every data flow that needs to exist in the target architecture, maps each to a Fabric pipeline or notebook, and produces the implementation backlog that Phase 7 will execute.

Your Challenge

Produce a complete catalogue of all data flows, pipelines, and notebooks required in the target Fabric architecture, with priority and estimated effort for each.

Things to Explore/Do

- Open `TravelReporting/Reports/Monthly/Display.cfm`, `Quarterly/Display.cfm`, and `Yearly/Display.cfm`. Ask Copilot: "*Analyse the SQL queries in these ColdFusion report files. What data do they aggregate? Map each query to a Fabric notebook transformation: which Azure SQL source table, what aggregation logic, and which Gold lakehouse table it should produce.*"
- Open `TravelReporting/Graphs/Monthly/Display.cfm` and `Graphs/Division/Display.cfm`. Ask Copilot the same question — identify what data drives each graph and map it to a Gold table.
- Ask Copilot: "*Design the complete Fabric pipeline topology for TravelReporting. I need: (1) a daily ingestion pipeline from Azure SQL to the Bronze lakehouse, (2) a Bronze→Silver transformation notebook, (3) a Silver→Gold aggregation notebook, (4) a semantic model refresh trigger. For each, specify: trigger type, source, destination, estimated runtime, and failure-alerting approach.*"
- Produce a **pipeline catalogue** as a Markdown table under `docs/pipeline-catalogue.md`:

Pipeline / Notebook	Source	Target	Trigger	Priority	Status
`pl_ingest_travel_daily`	Azure SQL `trre.TRAVEL`	Bronze `travel_raw`	Daily 02:00 AWST	High	To build
`nb_bronze_to_silver`	Bronze `travel_raw`	Silver `travel_clean`	After ingest	High	To build
`nb_silver_to_gold`	Silver `travel_clean`	Gold `travel_monthly_summary`	After silver	High	To build
`nb_silver_to_gold_division`	Silver `travel_clean`	Gold `travel_division_summary`	After silver	Medium	To build
`pl_refresh_semantic_model`	Gold lakehouses	Power BI dataset	After gold	High	To build

- Ask Copilot: "*Which of the ColdFusion report files in TravelReporting/Reports/ and TravelReporting/Graphs/ can be retired entirely once the Power BI reports from Phase 3 are live? Which ones have unique logic not yet covered by the semantic model?*"
- Identify any scheduled jobs or automated exports in the source (there are none found in this workspace — document this as a gap to fill).

What Done Looks Like

- ☐ `docs/pipeline-catalogue.md` lists all pipelines and notebooks with source, target, trigger, and priority
- ☐ Each ColdFusion report file is mapped to either a Power BI report page, a Gold table, or documented as "retire"
- ☐ The ingestion, transformation, and semantic model refresh sequence is fully designed
- ☐ No pipeline is marked "To build" without a clear owner and Phase 7 task linked to it

🌟 **Stretch Goal** — Ask Copilot to generate a Mermaid sequence diagram showing the full daily data pipeline execution flow: Azure SQL → Bronze → Silver → Gold → Power BI refresh, with error-handling branches and alert

triggers at each step.

Phase 6 — Performance Tuning & Index Optimisation

💬 Ask + 🤖 Agent mode

The ColdFusion application runs complex ad-hoc queries directly against Oracle with no query hints, no connection pooling strategy, and indexes defined separately in `TRRE-Oracle/INDEX.sql`. On Azure SQL, the query optimiser behaves differently, the `TRAVEL` table's 60-column wide row will benefit from careful index design, and the analytics queries running across months of data will need columnstore indexes on the Gold layer. This phase tunes the target schema before the ETL pipelines load real data.

Your Challenge

Review the top query patterns from the ColdFusion source, generate query plans for each, recommend indexes for Azure SQL and Fabric, and implement the highest-impact changes.

Things to Explore/Do

- Open `TravelReporting/Search/Select.cfm` — this is the most query-intensive page in the application. Ask Copilot: "*Analyse the SQL in TravelReporting/Search/Select.cfm. What columns does it filter and sort on? Generate the optimal non-clustered index or composite index for Azure SQL to support these query patterns on the TRAVEL table.*"
- Open `TravelReporting/Reports/Monthly/Display.cfm`. Ask Copilot: "*This ColdFusion page aggregates TRAVEL data by month and division. Generate the T-SQL query equivalent, then explain whether a non-clustered columnstore index on TRAVEL would improve this query and provide the DDL.*"
- Ask Copilot: "*The TRAVEL table in TRRE-Oracle/TABLE.sql has approximately 60 columns. What is the risk of table scans on this table in Azure SQL, and what covering index strategy would you recommend for the three most common access patterns: (1) search by STATUS + DEPART_DATE range, (2) lookup by ID, (3) aggregate by PERSON_ORG_CODE + month?*"
- Review `TRRE-Oracle/INDEX.sql` with Copilot: "*Translate these Oracle indexes to Azure SQL CREATE INDEX statements. Identify any that are redundant, any that should be columnstore, and any that are missing.*"
- For the Fabric Gold layer, ask Copilot: "*The Gold table gold.travel_monthly_summary will be queried by Power BI Direct Lake across 3+ years of monthly data. What Delta Lake optimisation settings should I apply (Z-ORDER, OPTIMIZE, VACUUM) and how frequently?*"
- Ask Copilot to generate a T-SQL script that queries `sys.dm_exec_query_stats` and `sys.dm_db_missing_index_details` after a test load and surfaces the top 10 missing indexes by estimated improvement.
- Store all index DDL under `database/indexes/`.

What Done Looks Like

- ☐ Top 5 query patterns from the ColdFusion source are documented with their Azure SQL equivalents
- ☐ Index recommendations are implemented in `database/indexes/` (at least 3 non-clustered indexes on TRAVEL)

- ☐ Columnstore index recommendation for the analytics query pattern is documented and implemented (or justified as deferred)
- ☐ Oracle index DDL from `TRRE-Oracle/INDEX.sql` is fully translated to Azure SQL
- ☐ Delta Lake optimisation strategy (Z-ORDER, OPTIMIZE schedule) for Gold tables is documented

🌟 **Stretch Goal** — Use the Azure MCP Server (`#mcp_azure_query_logs`) to query the Azure SQL Query Performance Insight after a seed-data test run and identify the top 3 queries by CPU time. Ask Copilot to generate the fix for each.

⏸️ GIT CHECKPOINT

```
git add database/ notebooks/ docs/ && git commit -m "phase-6: index strategy complete — Azure SQL
```

Phase 7 — Migration Automation (ETL/ELT) & DB CI/CD

🤖 Agent mode

Plans and designs become value only when they run automatically. This phase builds the actual Fabric pipelines and notebooks identified in Phase 5, and wires the Azure SQL schema into a DACPAC CI/CD pipeline so that every schema change is version-controlled, reviewed, and deployed consistently — replacing the current approach of running DDL scripts manually.

Your Challenge

Build the Bronze ingestion pipeline, the Bronze→Silver and Silver→Gold transformation notebooks, and a DACPAC-based Azure DevOps pipeline for Azure SQL schema deployments.

Things to Explore/Do

Fabric Data Pipeline — Bronze ingestion:

- Use `mcp_fabric_mcp_create_pipeline` or Agent mode: "*Generate a Fabric Data Pipeline JSON definition (`pl_ingest_travel_daily`) that copies all rows from Azure SQL table `trre.TRAVEL` to the Bronze lakehouse Delta table `travel_raw` , using a watermark column (UPDATE_DATETIME) for incremental load. Include error handling that writes failed rows to a `_errors` table.*"
- Ask Copilot to generate an incremental load notebook alternative: a PySpark cell that reads the high-watermark from a `pipeline_watermarks` control table, loads only new/updated rows, and updates the watermark after a successful run.

Fabric Notebooks — transformations:

- Use Agent mode: "*Generate a PySpark Fabric notebook called nb_bronze_to_silver with the following cells: (1) read bronze.travel_raw Delta table, (2) cast columns to correct types (DATE, DECIMAL, INT based on the TRRE-Oracle/TABLE.sql schema), (3) filter out rows where STATUS IS NULL, (4) deduplicate by ID keeping the latest UPDATE_DATETIME, (5) write to silver.travel_clean Delta table with MERGE (upsert on ID).*"

- Use Agent mode: "*Generate a PySpark Fabric notebook called nb_silver_to_gold that: reads silver.travel_clean, aggregates to the gold.travel_monthly_summary schema designed in Phase 4, and writes the result using overwrite mode on each partition (year, month).*"

- Ask Copilot to add a **data quality cell** to `nb_bronze_to_silver` that asserts the quality rules from Phase 4 (not null, date order, valid status) and writes a quality report row to `silver.data_quality_log`.

DB CI/CD with DACPAC:

- Use Agent mode: "*Generate an Azure DevOps pipeline YAML (`azure-pipelines-db.yml`) that: (1) runs `sqlpackage /Action:Script` to generate a deployment script from the DACPAC, (2) runs the script against the dev Azure SQL instance, (3) runs the validation scripts from database/validate/ as a post-deploy step, (4) gates promotion to production on manual approval.*"

- Ask Copilot: "*Generate a DACPAC project structure (`.sqlproj`) for the Azure SQL `trre` schema, importing the DDL from database/migrations/001_initial_schema.sql and the indexes from database/indexes/.*"

- Confirm that the DACPAC pipeline uses the Azure SQL connection string from Key Vault (co-ordinate with the platform track), not a hardcoded value.

What Done Looks Like

- ☐ `pl_ingest_travel_daily` pipeline loads from Azure SQL to Bronze with incremental watermark logic
- ☐ `nb_bronze_to_silver` notebook casts, deduplicates, and upserts to Silver Delta table
- ☐ `nb_silver_to_gold` notebook aggregates to `gold.travel_monthly_summary`
- ☐ `azure-pipelines-db.yml` deploys schema via DACPAC / `sqlpackage` with post-deploy validation
- ☐ No connection strings are hardcoded — all read from Key Vault or Fabric linked services
- ☐ All notebooks and pipelines are committed to the repository under `notebooks/` and `pipelines/`

🌟 **Stretch Goal** — Add a **pipeline monitoring notebook** that queries the Fabric pipeline run history via the Fabric REST API, computes the SLA compliance rate (% of daily runs that completed before 06:00 AWST), and posts a summary to a Teams webhook.

Phase 8 — Notebooks, Fabric Pipelines, TMDL Export, DR Scripts & Solution Docs

🤖 Agent mode

The final phase productionises everything: the Gold-layer notebooks get schedules and monitoring, the Power BI semantic model is exported as TMDL for source control, DR / restore scripts are written so the platform can be recovered from scratch, and the solution documentation is committed so the next engineer who inherits this platform does not have to reverse-engineer it.

Your Challenge

Finalise and schedule all Fabric pipelines and notebooks, export the semantic model as TMDL, write DR scripts, and produce the data architecture solution document.

Things to Explore/Do

Finalise and schedule Fabric pipelines:

- Use `mcp_fabric_mcp_*` or Agent mode to set schedules on `pl_ingest_travel_daily`: daily at 02:00 AWST (UTC+8).
- Add an **alerting step** to the pipeline: if the pipeline fails, send an email via Azure Monitor Action Group (co-ordinate with the platform track) or a Fabric alert.
- Add a **semantic model refresh** as the final step in the Gold pipeline: use the Fabric REST API or Power BI MCP to trigger a dataset refresh after the Gold notebooks complete.

TMDL export:

- Use `mcp_powerbi-model_export_tmdl` to export the semantic model to `powerbi/semantic-model/tmdl/` — verify the output contains `model.tmdl`, `tables/`, and `relationships/` files.
- Ask Copilot: "*Add a `.gitattributes` rule and a `README.md` to powerbi/semantic-model/tmdl/ explaining how to import this TMDL model back into Power BI Desktop or Fabric using the Power BI MCP.*"
- Confirm that the `.pbix` file (if Option B was chosen) is listed in `.gitignore` and the TMDL is the source-of-truth instead.

DR / restore scripts:

- Use Agent mode: "*Generate a PowerShell DR script (scripts/dr/restore-azure-sql.ps1) that: (1) uses the Azure CLI to restore the TravelReporting Azure SQL database from the latest automated backup to a new server, (2) runs the DACPAC to bring the schema to the latest version, (3) runs database/seeds/ scripts to restore reference data, (4) runs database/validate/ scripts to confirm integrity. Target RTO: 4 hours.*"
- Ask Copilot: "*Generate a PowerShell script (scripts/dr/restore-fabric-lakehouse.ps1) that uses the Fabric REST API to restore OneLake Delta tables from Azure Data Lake Storage Gen2 snapshots.*"

Solution documentation:

- Use Agent mode to generate `docs/data-architecture.md` covering:
  - Data estate overview (source Oracle schema → Azure SQL → Fabric Lakehouse → Power BI)
  - Medallion layer definitions (Bronze / Silver / Gold contracts — columns, types, quality rules)
  - Pipeline topology (Mermaid diagram from Phase 5, updated with final schedules)
  - Semantic model overview (tables, key relationships, key DAX measures)
  - DB CI/CD process (how schema changes are deployed)
  - DR process (RTO/RPO targets, recovery steps, contacts)
  - Known gaps and outstanding migration items
- Ask Copilot to generate `docs/data-dictionary.md` for `gold.travel_monthly_summary` and `silver.travel_clean` — column names, types, descriptions, and source mapping back to `TRRE.TRAVEL`.

What Done Looks Like

- ☐ All Fabric pipelines have schedules set and alerting configured
- ☐ Semantic model refresh is triggered automatically after Gold notebooks complete
- ☐ TMDL export exists under `powerbi/semantic-model/tmdl/` and is committed to the repository
- ☐ DR script for Azure SQL restore exists and is documented with tested RTO estimate
- ☐ DR script for Fabric lakehouse restore exists
- ☐ `docs/data-architecture.md` covers the full platform with Mermaid diagrams

- ☐ `docs/data-dictionary.md` covers the Gold and Silver key tables

🌟 **Stretch Goal** — Use `@azure-cost` to estimate the monthly Fabric capacity cost (F-SKU) for the pipeline schedule and data volumes involved. Document it in `docs/data-architecture.md` under a "Cost Optimisation" section with recommendations for autoscaling or pausing capacity outside business hours.

⏸️ GIT CHECKPOINT

```
git add notebooks/ pipelines/ powerbi/ scripts/dr/ docs/ && git commit -m "phase-8: platform prod
```

GHCP Prompting Cheat Sheet

💬 Ask Mode — Understanding & Reviewing

```
Open TRRE-Oracle/TABLE.sql. For the TRRE.TRAVEL table, list every Oracle-specific
data type, constraint, and feature used. For each one, tell me: (1) the Azure SQL
T-SQL equivalent, (2) any precision or behaviour difference I need to be aware of,
and (3) whether a code change is needed in the migrated .NET 8 application.
```

```
Open TravelReporting/Reports/Monthly/Display.cfm. Identify every SQL query on
this page. For each query: (1) describe what it computes, (2) write the equivalent
T-SQL for Azure SQL, (3) suggest which Gold lakehouse table in the medallion
architecture should pre-aggregate this result, and (4) identify which Power BI
DAX measure would replace the aggregation logic.
```

```
@azure-kusto Query the Log Analytics workspace for the TravelReporting Fabric
capacity unit. Show me pipeline run durations for the last 7 days, grouped by
pipeline name, so I can identify which notebooks are the slowest.
```

📋 Plan Mode — Designing

```
I am migrating the TRRE Oracle schema to Azure SQL and building a Microsoft Fabric
Bronze/Silver/Gold medallion architecture for TravelReporting analytics.

Source tables: TRAVEL (~60 cols), ACCOUNT_CODES, ARRANGER_GROUP, ARRANGER_USER,
APPROVER_USER, FLEX_COST_CODE, FLEX_PROJ_CODE, FUNDING_TYPE, SYSTEM_REF_VALUES.

Design the full Silver and Gold layer schemas. For Silver: what cleaning, type-casting,
and deduplication is needed? For Gold: what aggregated/denormalised tables does
```

```
Power BI need to serve the monthly, quarterly, yearly, and division reports currently
generated by TravelReporting/Reports/?
```

```
Design a DACPAC-based DB CI/CD strategy for the TravelReporting Azure SQL schema.
The pipeline runs in Azure DevOps. Requirements: (1) schema changes are authored
in a .sqlproj, (2) a deployment preview (drift report) runs before any apply,
(3) post-deploy validation scripts in database/validate/ run after every deploy,
(4) production deployment requires manual approval. Show the Azure DevOps stages
and the sqlpackage commands for each.
```

```
#mcp_fabric_mcp_list_lakehouses List all lakehouses in the TravelReporting-Analytics
Fabric workspace. For each lakehouse, show the Delta tables present and their
row counts. Identify any table that is empty and should have data from the
ingestion pipeline.
```

🤖 Agent Mode — Generating & Implementing

```
Generate a complete PySpark Fabric notebook (nb_bronze_to_silver) that:
1. Reads bronze_travel.travel_raw as a Delta table
2. Casts columns to the correct types based on TRRE-Oracle/TABLE.sql
   (TRAVEL table — NUMBER to DECIMAL/INT, VARCHAR2 to STRING, DATE to TIMESTAMP)
3. Filters out rows where STATUS IS NULL or DEPART_DATE IS NULL
4. Deduplicates by ID, keeping the row with the latest UPDATE_DATETIME
5. Writes to silver_travel.travel_clean using MERGE ON ID
6. Appends a data quality summary row to silver_travel.data_quality_log:
   run_date, rows_read, rows_written, rows_filtered, rows_deduplicated
Output as a .ipynb notebook file with one cell per step and markdown headers.
```

```
Generate DAX measures for a Power BI semantic model on the TravelReporting Azure SQL
schema. The TRAVEL table has: TOTAL_COST (estimated), ACT_TOTAL_COST (actual),
ACT_AIRFARE_COST, ACT_ACCOMMODATION_COST, ACT_ALLOWANCE_COST, ACT_CONFERENCE_COST,
DEPART_DATE, STATUS, PERSON_ORG_CODE, TRAVEL_TYPE.

Generate: Total Actual Cost, Cost Variance, Cost Variance %, YTD Actual Cost,
MoM Cost Change %, Avg Cost Per Trip, Airfare % of Total, Approved Travel Count,
Trips Pending Approval. Format each as a named DAX expression ready to paste
into Power BI Desktop or the Power BI MCP.
```

```
Generate an Azure DevOps pipeline YAML (azure-pipelines-db.yml) for DACPAC
deployment of the TravelReporting Azure SQL schema. Stages:
(1) Build: dotnet build the .sqlproj and publish the DACPAC artefact,
(2) Preview: sqlpackage /Action:Script to generate a drift report and publish it,
(3) Deploy-Dev: sqlpackage /Action:Publish to dev Azure SQL using service connection,
(4) Validate: run database/validate/validate-all.sql and fail if any FAIL output,
```

```
(5) Deploy-Prod: manual approval gate, then sqlpackage /Action:Publish to prod.
Read connection strings from variable group TravelReporting-DB-Secrets.
```

Final Deliverables Checklist

1. ☐ **Migration assessment + target Azure SQL DDL** — Risk matrix for all Oracle objects (`TRRE-Oracle/`), translated T-SQL DDL in `database/migrations/` , PL/SQL functions translated to T-SQL
2. ☐ **Test data + validation report** — FK-safe seed scripts in `database/seeds/` (50+ TRAVEL rows, all lookups), validation scripts in `database/validate/` with row count, FK, and business-rule assertions
3. ☐ **Power BI semantic model + DAX measures** — Connected to Azure SQL / Gold lakehouse, all 10 DAX measures implemented, at least 2 report pages (Monthly Cost Trend, Division Summary), TMDL exported or `.pbix` committed
4. ☐ **Fabric medallion layout** — Three lakehouses provisioned, Bronze/Silver/Gold table schemas documented in `docs/medallion-architecture.md` with Mermaid lineage diagram
5. ☐ **ETL / notebook catalogue** — `docs/pipeline-catalogue.md` listing all pipelines and notebooks with source, target, trigger, and priority; all ColdFusion reports mapped to Power BI pages or Gold tables
6. ☐ **Performance tuning report** — Top 5 query patterns analysed, index recommendations implemented in `database/indexes/` , Gold Delta Lake optimisation strategy documented
7. ☐ **Automated ETL + DB CI/CD pipelines** — `pl_ingest_travel_daily` , `nb_bronze_to_silver` , `nb_silver_to_gold` implemented; `azure-pipelines-db.yml` deploys via DACPAC with post-deploy validation; no hardcoded secrets
8. ☐ **TMDL export + DR scripts + solution docs** — TMDL in `powerbi/semantic-model/tmdl/` , DR scripts in `scripts/dr/` , `docs/data-architecture.md` with full Mermaid diagrams, `docs/data-dictionary.md` for Silver and Gold key tables

---

Tips for Success

1. **Source-control TMDL, not `.pbix` .** A `.pbix` file is binary — it cannot be meaningfully diff'd, reviewed in a PR, or merged. Export your semantic model as TMDL using the Power BI MCP and commit those text files. Your Power BI model is code.
2. **Validate row counts AND checksums.** Row counts can match while data is wrong. After every migration run, compare both the count and a checksum (SHA-256 of sorted key columns) between source and target. Phase 2 gives you the scripts to do this.
3. **Prefer MCP-driven authoring over the portal.** Creating lakehouses, pipelines, and semantic models through the Fabric and Power BI MCPs keeps a record in your Git history. Portal clicks leave no audit trail and cannot be replicated in a CI/CD pipeline.
4. **Treat notebooks as code.** PySpark notebooks are `.ipynb` files — commit them to `notebooks/` , review them in PRs, and run them in the DACPAC pipeline's post-deploy stage. A notebook that only exists in the Fabric portal is a deployment risk.

5. **Translate Oracle DECODE early.** Every `DECODE` in the ColdFusion queries will become a `CASE WHEN` in T-SQL. If you leave this until late, you will find it scattered throughout the Silver and Gold transformation notebooks. Centralise the translation in Phase 1 and the rest of the migration is much cleaner.
6. **Co-ordinate with the platform track on Key Vault.** The Azure SQL connection string, Fabric linked service credentials, and Power BI dataset refresh tokens all belong in Key Vault. Agree the secret names early so your Fabric pipelines and the platform track's smoke scripts reference the same names.
7. **Use seed data from Day 1.** Phase 2 generates seed data specifically so Phases 3, 4, and 5 have something real to work with. Load it into Azure SQL at the end of Phase 2 — every subsequent phase will be more productive with data in the system.
8. **Partial completion is fine.** A working Bronze → Silver pipeline with a connected Power BI report is a better demonstration than a fully designed but unbuilt Gold layer. Deliver end-to-end value on one slice, then expand coverage.